Bitácora de programación con IA

Proyecto: **Clima Claro** · Autor: **Angel Guadalupe Balam Moo** · Herramienta principal: **OpenAI Codex**

Contexto del proyecto

Se desarrolló una aplicación web que usa las API de geocodificación y pronóstico de Open-Meteo. La aplicación permite buscar una ciudad, consultar el clima actual y ver un pronóstico de siete días. Codex se utilizó para analizar la consigna, escribir y depurar el código, preparar pruebas y documentar el proyecto. Las dos últimas entradas de esta bitácora son instrucciones textuales de la consigna adjunta que se usaron como subprompts de trabajo; no se presentan como mensajes escritos por separado.

Prompts e iteraciones

1. Análisis del encargo

> ***Prompt:*** *“Analiza el documento y dime que es lo que entiendes que hay que hacer”*

Respuesta obtenida: La IA separó la actividad en aplicación, repositorio, README, bitácora, evidencia de GitHub y demostración. También señaló las condiciones de seguridad para las claves.

Resultado y ajustes: Sí funcionó a la primera. Solo fue necesario confirmar que la demostración sería responsabilidad del estudiante.

2. Inicio de la implementación

> ***Prompt:*** *“Bien, Sigue las instrucciones del documento excepto el inciso 6, pues ese me tengo que encargar yo”*

Respuesta obtenida: La IA eligió una aplicación web de clima con Open-Meteo, una API gratuita que no requiere clave, y propuso dividir el desarrollo en varios commits.

Resultado y ajustes: La instrucción fue suficiente para iniciar, pero se revisó primero la identidad de Git para evitar que los commits quedaran atribuidos a otra persona.

3. Confirmación del alcance

> ***Prompt:*** *“Esta bien, Bien, Sigue las instrucciones del documento excepto el inciso 6, pues ese me tengo que encargar yo”*

Respuesta obtenida: La IA implementó la interfaz, las peticiones a la API, los estados de error, las pruebas y la documentación, manteniendo fuera del alcance el video o presentación.

Resultado y ajustes: No se necesitó cambiar el objetivo. Se mejoraron los textos en español y se añadió un botón de reintento para que los errores fueran recuperables.

4. Requisito textual usado como subprompt

> ***Prompt:*** *“Desarrolla una aplicación pequeña (web o móvil) que consuma al menos una API externa. Puedes elegir la API y el tema.”*

Respuesta obtenida: La respuesta fue una app web sin framework llamada Clima Claro. Se usó una API de geocodificación para obtener coordenadas y otra de pronóstico para los datos del clima.

Resultado y ajustes: La primera propuesta funcional necesitó validación de la estructura de respuesta y mensajes claros cuando una ciudad no existe.

5. Requisito textual usado como subprompt

> ***Prompt:*** *“Manejar errores (sin conexión, respuesta vacía, límite de peticiones).”*

Respuesta obtenida: La IA creó mensajes distintos para falta de conexión, tiempo agotado, respuesta vacía, límite HTTP 429 y errores generales. También añadió reintento y detección de cambios de red.

Resultado y ajustes: No quedó completo a la primera: una prueba falló porque el mensaje decía 'limite' sin acento mientras la prueba esperaba 'límite'. Se corrigió el texto y después pasaron las seis pruebas.

Error de la IA detectado

Durante la primera ejecución de las pruebas automatizadas, una prueba falló por una inconsistencia entre el texto esperado y el texto generado para el límite de peticiones. El programa mostraba “limite” y la prueba exigía “límite”. Aunque no impedía la petición a la API, demostraba que el resultado de la IA no debía aceptarse sin revisión. Se corrigió la cadena en `weather-utils.js` y se volvió a ejecutar `npm test`; las seis pruebas pasaron.

Qué aprendí

Aprendí que una petición clara mejora mucho el resultado, pero no sustituye la verificación. También aprendí a separar la lógica en funciones pequeñas, validar los datos antes de mostrarlos y tratar cada error con un mensaje útil. Consultar la documentación oficial de la API evitó depender de parámetros inventados o desactualizados.

Qué haría distinto la próxima vez

La próxima vez dividiría desde el inicio el trabajo en prompts más específicos: uno para la estructura, otro para la API, otro para errores y otro para pruebas. También pediría criterios de aceptación antes de generar código y realizaría una prueba manual después de cada cambio importante, no únicamente al final.

Verificación final	6 pruebas automatizadas aprobadas · claves expuestas: ninguna · historial: varios commits